Assignment :- 2

Name:- Gyan Prakash Tiwari

Roll No.:- 202401100200146

Branch:- CSE “5B”

LAB 3: Identify the classes. Classify them as weak and strong classes and draw the class diagram.

UML Class Diagram – AI Chatbot for University Helpdesk

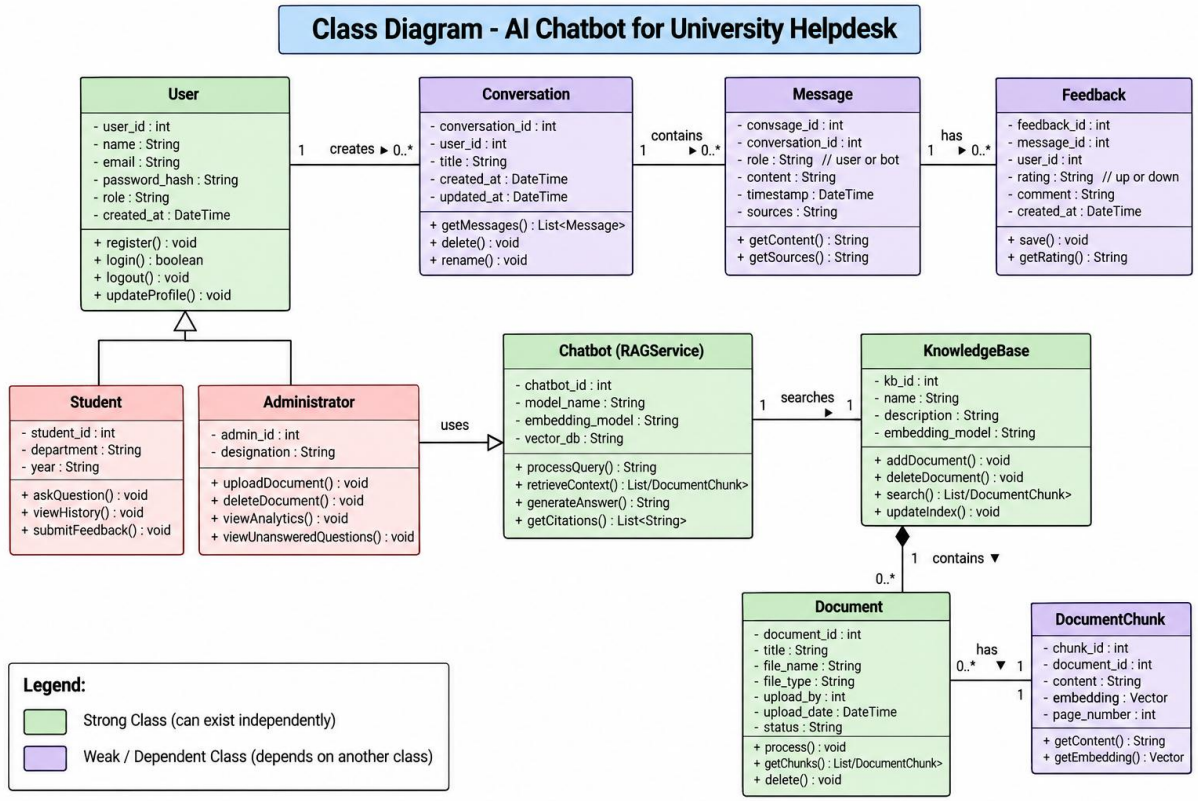
UML Class Diagram

A UML Class Diagram for the **AI Chatbot for University Helpdesk** represents the static structure of the system by displaying its classes, attributes, methods, and relationships. It helps in understanding how different components such as users, conversations, messages, documents, the knowledge base, and the AI chatbot interact with each other.

The class diagram:

- Represents the classes, attributes, methods, and relationships of the AI University Helpdesk system.
- Helps communicate and document the software architecture.
- Shows how students, administrators, chatbot services, conversations, and university documents are connected.

- Makes it easier for developers and designers to understand and implement the system.
- Helps identify strong and weak/dependent classes in the system.



UML Class Notation

Classes in the **AI Chatbot for University Helpdesk** are represented using rectangular boxes divided into three compartments: **Class Name, Attributes, and Methods**.

1. Class Name

The class name is written in the top compartment of the class box and represents an entity or component of the system.

The major classes identified in the project are:

- User
- Student
- Administrator
- Conversation
- Message
- Feedback
- Chatbot/RAGService
- KnowledgeBase
- Document
- DocumentChunk

---

2. Attributes

Attributes represent the data or properties maintained by a class. They are written in the second compartment of the class.

For example, the **User** class contains attributes such as:

- userId : int
- name : String
- email : String
- passwordHash : String
- role : String

Similarly, the **Document** class contains attributes such as:

- documentId : int
- fileName : String
- fileType : String
- uploadedAt : DateTime
- uploadedBy : User

Attributes describe the current state and information associated with an object in the system.

---

3. Methods

Methods represent the operations or functions that can be performed by a class. They are written in the third compartment of the class box.

For example, the **User** class provides methods such as:

- register()
- login()
- logout()

The **Chatbot/RAGService** class provides methods such as:

- askQuestion()
- retrieveContext()
- generateAnswer()
- citeSources()

The **Administrator** class provides operations such as:

- uploadDocument()

- deleteDocument()
- viewAnalytics()
- viewUnansweredQuestions()

These methods define the behavior of different objects within the AI University Helpdesk system.

---

4. Visibility Notation

Visibility notation specifies the accessibility of attributes and methods in the classes.

The commonly used UML visibility symbols are:

- **+** for Public — accessible from other classes.
- **-** for Private — accessible only inside the class.
- **#** for Protected — accessible within the class and its subclasses.
- **~** for Package/Default — accessible to classes belonging to the same package.

For example:

User

- userId : int

- name : String

- email : String

- passwordHash : String

+ register() : void

+ login() : boolean

+ logout() : void

Here, the attributes are private while the operations are public.

Parameter Directionality

Parameter directionality represents the direction in which information is transferred between methods and objects.

The three main parameter directionality types are:

In (Input)

An input parameter passes information from the caller to a method.

For example:

askQuestion(question : String)

Here, question is provided by the student to the chatbot.

Out (Output)

An output parameter represents information returned by a method.

For example:

generateAnswer() : String

The generated answer is returned by the chatbot to the user.

InOut (Input and Output)

An InOut parameter can be passed to a method, modified during processing, and returned to the caller.

For example, conversation information may be received, updated, and returned while processing a chatbot interaction.

Identification of Classes

The following classes are identified for the **AI Chatbot for University Helpdesk**:

Class	Description
User	Represents a general registered user of the system.
Student	Represents a student who interacts with the AI chatbot.
Administrator	Manages documents, knowledge base, analytics, and unanswered questions.
Conversation	Stores a conversation between a student and the chatbot.
Message	Represents an individual user or chatbot message.
Feedback	Stores feedback submitted for chatbot responses.

Class	Description
Chatbot/RAGService	Processes student queries and generates AI responses.
KnowledgeBase	Stores and manages searchable university information.
Document	Represents university documents uploaded by administrators.
DocumentChunk	Represents smaller indexed sections of uploaded documents.

Strong Classes

Strong classes can exist independently and represent major entities or services in the system.

The strong classes identified in the project are:

1. User

The User class represents a registered user and contains basic authentication and profile information.

2. Document

The Document class represents university documents uploaded to the system.

3. KnowledgeBase

The KnowledgeBase class manages university information used by the chatbot for information retrieval.

4. Chatbot/RAGService

The Chatbot/RAGService class handles queries, retrieves relevant information, and generates responses.

Weak / Dependent Classes

Weak classes depend on other classes for their existence or meaningful operation.

1. Conversation

A Conversation depends on a User because conversations are created as a result of a user's interaction with the chatbot.

2. Message

A Message depends on a Conversation because every message belongs to a particular conversation.

3. Feedback

Feedback depends on the User and chatbot Message because it represents a user's evaluation of a generated response.

4. DocumentChunk

A DocumentChunk depends on a Document because it represents a smaller portion of an uploaded document used for information retrieval.

Relationships Between Classes

Relationships in the class diagram describe how different components of the **AI Chatbot for University Helpdesk** interact with one another.

The major relationships used in the project are:

1. Association

Association represents a relationship between two classes.

Examples in the project include:

- User is associated with Conversation.
- User is associated with Feedback.
- Administrator is associated with Document.
- Student is associated with Chatbot.

A student interacts with the chatbot to ask university-related questions, while an administrator interacts with documents and the knowledge base.

---

2. Directed Association

A directed association represents a relationship where one class accesses or interacts with another class in a particular direction.

For example:

Student → Chatbot/RAGService

The Student uses the Chatbot/RAGService to submit questions and receive responses.

Another example is:

Administrator → Document

The Administrator manages university documents.

---

3. Aggregation

Aggregation represents a whole-part relationship in which the contained object can still exist independently.

In the AI University Helpdesk, the **KnowledgeBase** can be associated with multiple **Documents**.

KnowledgeBase ◇—— Document

The KnowledgeBase organizes and manages information obtained from university documents.

---

4. Composition

Composition represents a strong whole-part relationship where the child entity depends on the parent entity for its existence.

Examples in the project include:

Conversation ◆—— Message

A Message belongs to a Conversation and is meaningful within that conversation.

Another important relationship is:

Document ◆—— DocumentChunk

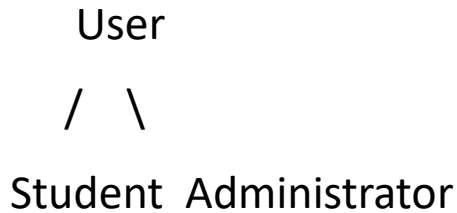
A DocumentChunk is generated from a Document and depends on that document.

---

5. Generalization (Inheritance)

Generalization represents an **"is-a"** relationship.

In this project, **Student** and **Administrator** are specialized types of **User**.



Therefore:

Student ————|> User

Administrator –|> User

Student and Administrator inherit common properties such as name, email, password, and authentication functionality from User while providing their own specialized operations.

---

6. Realization (Interface Implementation)

Realization is used when a class implements functionality defined by an interface.

If interfaces are introduced in the implementation, services such as the Chatbot/RAGService can implement interfaces defining operations such as information retrieval or response generation.

Realization is represented using a dashed line with a hollow arrowhead.

---

7. Dependency Relationship

A dependency exists when one class requires another class to perform its functionality.

An important dependency in the project is:

Chatbot/RAGService → KnowledgeBase

The Chatbot/RAGService depends on the KnowledgeBase to retrieve relevant university information before generating an answer.

The chatbot therefore uses information retrieved from the knowledge base as context for generating a response.

---

8. Usage (Dependency) Relationship

A usage relationship occurs when one class uses the functionality provided by another class without owning that class.

For example:

Chatbot/RAGService -----> KnowledgeBase

The Chatbot/RAGService uses the KnowledgeBase to search for relevant information.

Similarly:

Administrator -----> KnowledgeBase

The Administrator uses knowledge-base functionality while managing university documents.

Important Class Relationships in the Project

The main relationships can be summarized as:

Student ————|> User

Administrator —|> User

User ———————— Conversation

Conversation —— Message

Message ————— Feedback

User ———————— Feedback

Student ————— Chatbot/RAGService

Chatbot/RAGService ——> KnowledgeBase

Administrator ————> Document

KnowledgeBase ————— Document

Document ————————— DocumentChunk

A User can have multiple Conversations. A Conversation can contain multiple Messages. A Message may receive Feedback. Administrators manage university Documents, while the KnowledgeBase maintains information extracted from those

documents. The Chatbot/RAGService retrieves relevant information from the KnowledgeBase to generate answers for students.

Purpose of the Class Diagram

The main purposes of the class diagram for the **AI Chatbot for University Helpdesk** are:

- To represent the static structure of the application.
- To identify important classes in the system.
- To specify attributes and methods associated with each class.
- To distinguish between strong and weak/dependent classes.
- To represent relationships among different system components.
- To show inheritance between User, Student, and Administrator.
- To show how conversations and messages are organized.
- To represent how university documents are stored and processed.
- To understand how the RAG-based chatbot interacts with the knowledge base.
- To provide a blueprint for implementing the application.

Benefits of the Class Diagram

The class diagram provides the following benefits for the **AI Chatbot for University Helpdesk**:

- It provides a clear visual representation of the software architecture.
- It helps developers understand the major entities in the application.
- It represents attributes and methods associated with each entity.
- It identifies relationships such as association, composition, inheritance, and dependency.
- It makes the interaction between Student, Administrator, Chatbot, KnowledgeBase, and Document components easier to understand.
- It helps in designing the backend database and application models.
- It provides guidance during implementation and testing.
- It improves communication between developers and project stakeholders.
- It makes future modifications and extensions to the system easier.
- It provides proper documentation of the object-oriented design of the project.

Conclusion

The UML Class Diagram of the **AI Chatbot for University Helpdesk** provides a structural representation of the system and its major components. It identifies important classes such as **User, Student, Administrator, Conversation, Message, Feedback, Chatbot/RAGService, KnowledgeBase, Document, and DocumentChunk**.

The diagram also represents relationships such as **association, inheritance, composition, aggregation, and dependency**. By separating strong and weak/dependent classes and showing how the classes interact, the class diagram provides a clear blueprint for designing and implementing the AI-powered university helpdesk system.